Step 9 — 3-Gene GNN: What We Did and Why

Goal

PI direction: scale exact DP (backward induction) to 3 genes and train a GNN to approximate V*. Previous work (steps 1–8) covered 1–2 genes and single pedigree structures. This step extends to: - 3 genes (GeneA, GeneB, GeneC) - 3 pedigree structures (Trio N=3, Nuclear N=4, ThreeGeneration N=5) - 6 allele frequency regimes × 2 cost presets = 36 dataset configs

---

Problem Setup: Sequential Genetic Testing POMDP

At each step, a clinician decides which family member to test next (or stop). A state `s` = frozenset of (person, genotype_tuple) pairs already tested. The exact DP (backward induction) computes V*(s) for every reachable state.

3-Gene State Space

Family	N (people)	States per config	Method
Trio	3	4,872	Exact DP
Nucl	4	43,904	Exact DP
3Gen	5	1,054,528	Exact DP

ThreeGeneration has 1M+ states per config — exact DP is tractable but GNN generalization across new allele freq regimes is the goal (avoid rerunning DP for every new parameter set).

GNN Architecture (`shared/model.py` → `PedigreeGNN`)

Node Features (10 per person)

```
[P(g=0)_GeneA, P(g=1)_GeneA, P(g=2)_GeneA,
 P(g=0)_GeneB, P(g=1)_GeneB, P(g=2)_GeneB,
 P(g=0)_GeneC, P(g=1)_GeneC, P(g=2)_GeneC,
 is_tested]
```

`node_feat_dim = 3 * 3_genes + 1 = 10`

Architecture

- 2 rounds of message passing along pedigree edges (parent→child)
- Per round: `msg MLP(src_feat || dst_feat → 32)` + `upd MLP(node_feat || agg_msg → 32)`
- After 2 rounds: per-node embeddings are (B, N, 32)

- Global mean pool → (B, 32) graph embedding
- Head MLP: 32 → 16 → 1 (predicts scalar V*(s))

Key design: handles variable N

Mean pooling means the same GNN works for N=3, 4, 5 nodes — trained on all three families simultaneously with each family's own edge_index.

Dataset Construction (`step9_gnn_3gene/build_datasets.py`)

Pedigree Structures (`shared/data_gen.py` → `FAMILY_CASES`)

```
"Trio":             [("Child", "Parent1", "Parent2")]
"Nuclear":          [("Child1", "Parent1", "Parent2"), ("Child2", "Parent1", "Parent2")]
"ThreeGeneration":  [("Father", "Grandfather", "Grandmother"), ("Child", "Father", "Mother")]
```

Allele Frequency Regimes

```
LowHigh:    GeneA=0.02, GeneB=0.15, GeneC=0.10
MediumEven: GeneA=0.08, GeneB=0.08, GeneC=0.08
LowLow:     GeneA=0.02, GeneB=0.02, GeneC=0.02
HighHigh:   GeneA=0.20, GeneB=0.20, GeneC=0.20
MixedA:     GeneA=0.02, GeneB=0.08, GeneC=0.15
MixedB:     GeneA=0.15, GeneB=0.08, GeneC=0.02
```

Cost Presets

- Base: a=-0.05, b=-0.025, delta=0.75 (for GeneA/GeneB; GeneC: PLACEHOLDER values)
- Aggressive: a=-0.075, b=-0.0375, delta=0.80

NOTE: GeneC parameters are placeholders — not yet confirmed by PI.

Train/Test Split

- 30 train configs (5 regimes × 2 presets × 3 families)
- 6 test configs held out: `MediumEven_Aggressive` + `HighHigh_Aggressive` for each family

Training (`step9_gnn_3gene/run.py`)

Fast Data Loading

Each pkl stores pre-computed X array of shape `(N_states, n_people * 9)`. We reshape `(N, n_people*9)` → `(N, n_people, 9)` and append tested flag column → `(N, n_people, 10)`. This avoids calling `state_to_graph()` 3M times (was 53 min; now ~3 min).

Multi-Topology Training

Three family groups each epoch, each with own `edge_index`. Train/val split uses contiguous slices (not random index — avoids GPU memory copies). Batched val forward (batch_size=512) to avoid OOM on 2.1M ThreeGeneration val samples.

Training Stats (epoch 50 / 500)

- Total training samples: 11,033,040 (Trio=48,720, Nuclear=439,040, ThreeGen=10,545,280)
- train_loss=1.53e-5, val_loss=4.58e-5
- Checkpoint: `results/gnn3_ckpt.pt` (saved every 50 epochs)
- SLURM wall hit at 4h after epoch 50

Known Issue: Class Imbalance

```
Trio:       48,720 samples =  0.44% of training data
Nuclear:   439,040 samples =  3.98%
ThreeGen:10,545,280 samples = 95.58%
```

The GNN is almost entirely optimized for ThreeGeneration. Trio and Nuclear receive proportionally very little gradient signal. This inflates ratio2 for Trio at small V_root - V_stop gaps.

Evaluation (`step9_gnn_3gene/eval.py`)

Metric: ratio2

```
ratio2 = (V_root - L) / (V_root - V_stop)
```

- `V_root`: exact DP optimal value from root state (from pkl)
- `V_stop`: value of stopping immediately at root
- `L`: value achieved by greedy GNN policy (recursive rollout using GNN as lookahead)
- 0 = GNN is optimal, 1 = GNN no better than stopping

How eval works

1. Batch-infer GNN V*(s) for all states in one forward pass → dict {state: value}
2. Greedy rollout from root: at each state, pick action = person i that maximizes r_i + E[GNN(s')]
3. Compare result L to V_root and V_stop

Results (epoch 50 / 500, no class balancing)

All 36 Configs (sorted by ratio2)

Split	Config	ratio2	V_root	V_stop	L
TRAIN	Trio_LowLow_Base	1.000000	-0.032755	-0.043748	-0.043748
TRAIN	Trio_LowHigh_Base	0.120781	-0.085160	-0.144153	-0.092285
TRAIN	Trio_MixedA_Base	0.099484	-0.078971	-0.130956	-0.084143
TRAIN	Trio_MixedB_Base	0.065461	-0.097611	-0.166083	-0.102094
TRAIN	Trio_MediumEven_Base	0.054671	-0.086578	-0.154106	-0.090270
TRAIN	Nucl_LowLow_Aggr	0.042527	-0.048883	-0.098201	-0.050981
TRAIN	3Gen_MixedB_Base	0.021375	-0.163278	-0.290645	-0.166000
TRAIN	Nucl_MixedA_Aggr	0.018921	-0.112305	-0.291531	-0.115696
TEST	**Nucl_MediumEven_Aggr**	**0.018252**	-0.124906	-0.343236	-0.128891
TEST	**Trio_MediumEven_Aggr**	**0.016480**	-0.091095	-0.228824	-0.093365
TRAIN	Nucl_MixedB_Aggr	0.015831	-0.142454	-0.366833	-0.146006
TRAIN	Nucl_LowHigh_Aggr	0.014481	-0.120209	-0.319261	-0.123091
TRAIN	Trio_MixedA_Aggr	0.013885	-0.082780	-0.194354	-0.084329
TRAIN	Trio_MixedB_Aggr	0.012651	-0.102889	-0.244555	-0.104681
TRAIN	Trio_LowHigh_Aggr	0.011718	-0.088195	-0.212841	-0.089655
TRAIN	3Gen_MixedB_Aggr	0.010725	-0.172544	-0.427972	-0.175283
TEST	**Trio_HighHigh_Aggr**	**0.010632**	-0.171375	-0.429754	-0.174123
TRAIN	Nucl_MixedB_Base	0.009194	-0.134518	-0.249124	-0.135572
TRAIN	3Gen_LowHigh_Base	0.007827	-0.141294	-0.252268	-0.142162
TRAIN	3Gen_LowLow_Aggr	0.007034	-0.062950	-0.114568	-0.063313
TRAIN	3Gen_HighHigh_Base	0.006397	-0.272585	-0.516348	-0.274145
TRAIN	Trio_HighHigh_Base	0.006088	-0.160816	-0.295056	-0.161633
TRAIN	Nucl_LowHigh_Base	0.005771	-0.114850	-0.216230	-0.115436
TRAIN	3Gen_MixedA_Base	0.004502	-0.130632	-0.229172	-0.131076
TRAIN	Nucl_MixedA_Base	0.004195	-0.105324	-0.196434	-0.105706
TRAIN	3Gen_MediumEven_Base	0.004103	-0.143984	-0.269686	-0.144500
TRAIN	3Gen_LowHigh_Aggr	0.003618	-0.146673	-0.372471	-0.147490
TRAIN	Nucl_HighHigh_Base	0.003366	-0.224661	-0.442584	-0.225395
TRAIN	3Gen_MixedA_Aggr	0.003305	-0.137365	-0.340119	-0.138035
TEST	**Nucl_HighHigh_Aggr**	**0.003067**	-0.242412	-0.644630	-0.243646
TEST	**3Gen_MediumEven_Aggr**	**0.002692**	-0.151915	-0.400441	-0.152584
TRAIN	Nucl_LowLow_Base	0.001176	-0.046913	-0.065623	-0.046935
TEST	**3Gen_HighHigh_Aggr**	**0.001164**	-0.291012	-0.752069	-0.291549
TRAIN	Nucl_MediumEven_Base	0.001086	-0.117990	-0.231159	-0.118113
TRAIN	Trio_LowLow_Aggr	0.000027	-0.038810	-0.065467	-0.038811
TRAIN	3Gen_LowLow_Base	0.000004	-0.056015	-0.076560	-0.056015

Notable anomaly: Trio_LowLow_Base (TRAIN) ratio2=1.0

- V_root=-0.0328, V_stop=-0.0437: testing margin is only 0.011 (very rare disease)

- GNN policy stopped immediately at root (L = V_stop)
- Root cause: combination of tiny testing margin + Trio only 0.44% of training data
- Not a code bug — a class imbalance + underfitting issue

Summary by family

- **ThreeGeneration**: excellent (test ratio2 <= 0.003), trained on most data
- **Nuclear**: good (test ratio2 <= 0.018), 4% of training data
- **Trio**: noisy (test ratio2 <= 0.017, one train config = 1.0), only 0.44% of training data

---

Files

```
shared/model.py                          # GNN architecture (PedigreeGNN)
shared/data_gen.py                       # Pedigree family definitions + dataset builder
step9_gnn_3gene/build_datasets.py        # Builds 36 pkl files via exact DP
step9_gnn_3gene/run.py                   # Multi-topology GNN training
step9_gnn_3gene/eval.py                  # Batch inference + ratio2 computation
step9_gnn_3gene/results/gnn3_model.pt    # Trained weights (epoch 50)
step9_gnn_3gene/results/gnn3_ckpt.pt     # Full checkpoint (model + history)
step9_gnn_3gene/results/cache/           # 36 pkl files (exact DP datasets)
step9_gnn_3gene/results/partial_eval.json  # Full ratio2 results (36 configs)
```

Next Steps

1. Confirm GeneC parameters with PI (currently placeholder values)
2. Fix class imbalance: per-family loss weighting or cap ThreeGeneration samples
3. Train to convergence (500 epochs) with balanced data
4. Consider reporting ratio2 per family separately to PI